

Diagnóstico de Falhas em Rolamentos com CWT + CNN

Transformada Wavelet Contínua e Redes Neurais Convolucionais aplicadas ao diagnóstico de falhas em rolamentos industriais.



Sumário

- [Visão geral](#)
 - [Justificativa técnica: CWT vs. FFT vs. STFT](#)
 - [Escalogramas por classe](#)
 - [Pipeline](#)
 - [Resultados](#)
 - [Estrutura do repositório](#)
 - [Instalação](#)
 - [Uso](#)
 - [Modelos de Transfer Learning](#)
 - [Base teórica](#)
 - [Roadmap](#)
 - [Autor](#)
 - [Licença](#)
-

Visão geral

Rolamentos são responsáveis por mais de 30% das falhas em máquinas elétricas rotativas, o que torna o diagnóstico precoce um requisito central de manutenção preditiva industrial. Este projeto implementa um pipeline de diagnóstico de falhas que combina:

- **Transformada Wavelet Contínua (CWT)** para converter sinais de vibração unidimensionais em representações tempo-frequência bidimensionais (escalogramas), capturando a natureza não estacionária das falhas;
- **Redes Neurais Convolucionais (CNN)** treinadas do zero e via Transfer Learning (ResNet18, Inception-v3, EfficientNet-B0) para classificação automática dos escalogramas gerados.

O trabalho utiliza o dataset público **CWRU** (Case Western Reserve University), benchmark consolidado na literatura de diagnóstico de falhas em rolamentos, contemplando condições de operação normal e falhas na pista interna, pista externa e elemento rolante.

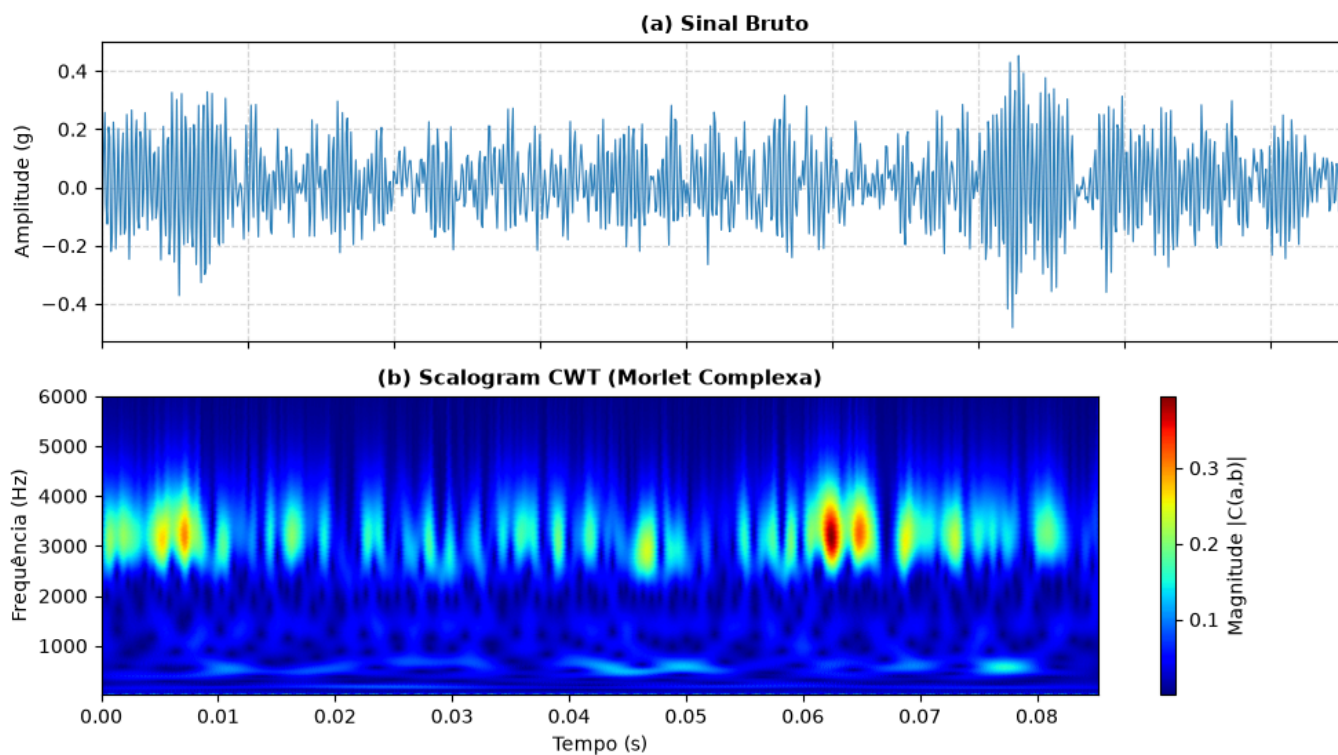
O relatório técnico completo (fundamentação teórica, metodologia e resultados) estará futuramente disponível em [docs/TCC.pdf](#).

Justificativa técnica: CWT vs. FFT vs. STFT

Característica	FFT	STFT	CWT
Domínio de análise	Frequência	Tempo-frequência	Tempo-frequência
Tipo de janela	Sinal inteiro	Fixa	Adaptativa
Resolução temporal	Nenhuma	Fixa	Variável
Resolução em frequência	Fixa	Fixa	Variável
Adequação a sinais não estacionários	Inadequada	Parcial	Adequada

Falhas em rolamentos geram sinais não estacionários, com eventos transitórios que a FFT não consegue localizar no tempo. A CWT contorna essa limitação ao adaptar dinamicamente sua janela de análise, janelas estreitas para altas frequências (maior resolução temporal) e janelas largas para baixas frequências (maior resolução espectral).

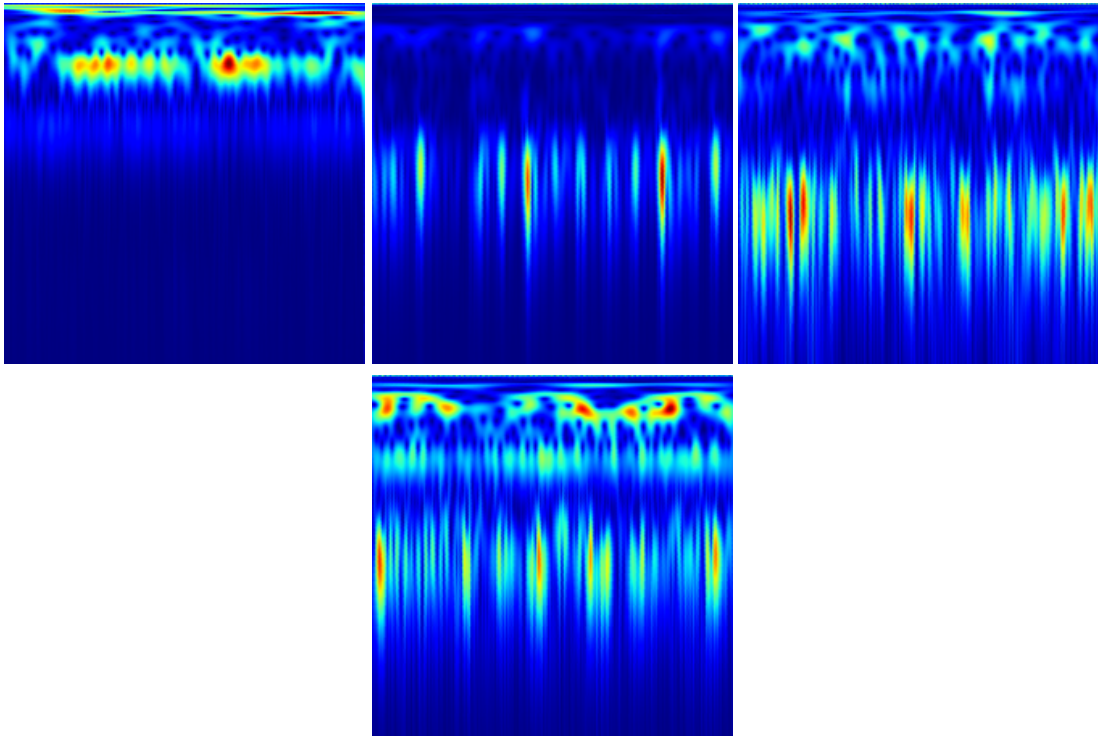
Diagnóstico de Falha no Rolamento — Esfera (Ball)



Sinal de vibração bruto no domínio do tempo e escalograma correspondente, obtido pela CWT com a wavelet-mãe de Morlet.

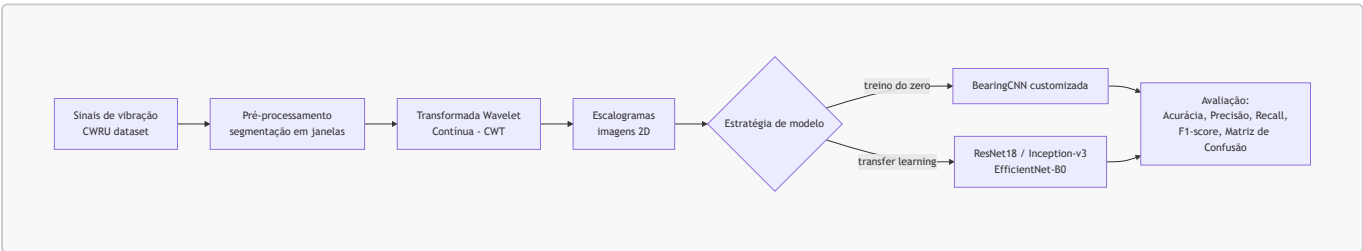
Escalogramas por classe

Cada classe de falha produz uma assinatura característica no escalograma, o que torna o problema adequado à classificação por CNN.



Da esquerda para a direita: condição normal, falha na pista interna, falha na pista externa, falha no elemento rolante.

Pipeline



Resultados

Resultados obtidos nos testes em 2.368 amostras inéditas do conjunto de teste, com **divisão estrita por arquivos .mat** (prevenindo *Data Leakage* por sobreposição de janelas) e checkpointing automático pelo menor `val_loss`:

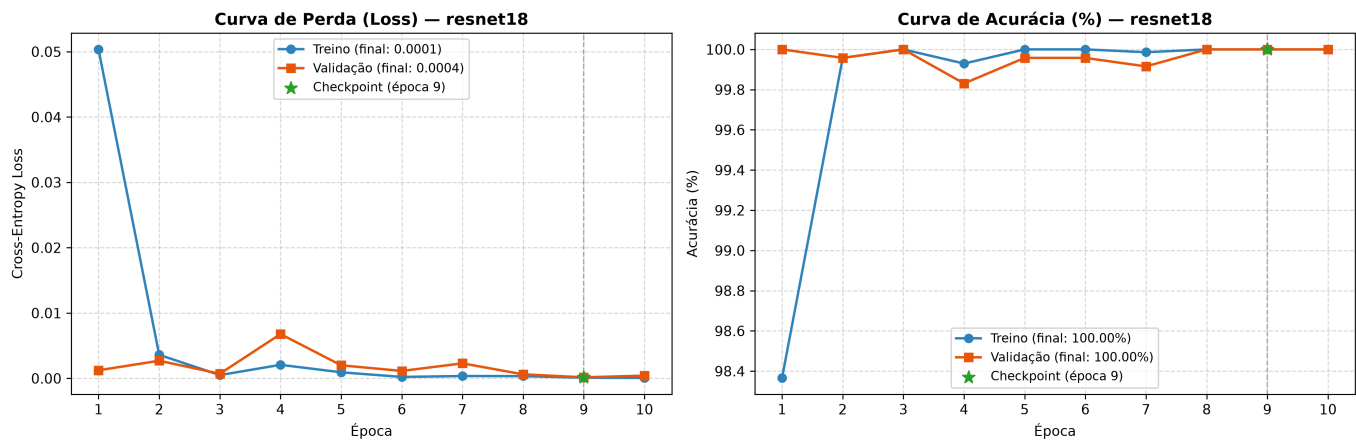
Modelo	Estratégia	Acurácia Multiclasse	Detecção Binária (Normal vs. Falha)	Parâmetros
BearingCNN (Própria)	Treino do zero	97.59%	100.00%	~1.2M
ResNet18	Fine-tuning (Freeze=False)	99.83%	100.00%	~11.1M
EfficientNet-B0	Fine-tuning (Freeze=False)	98.48%	100.00%	~5.3M

Modelo	Estratégia	Acurácia Multiclasse	Detecção Binária (Normal vs. Falha)	Parâmetros
Inception-v3	Fine-tuning (Freeze=False)	97.93%	100.00%	~23.8M

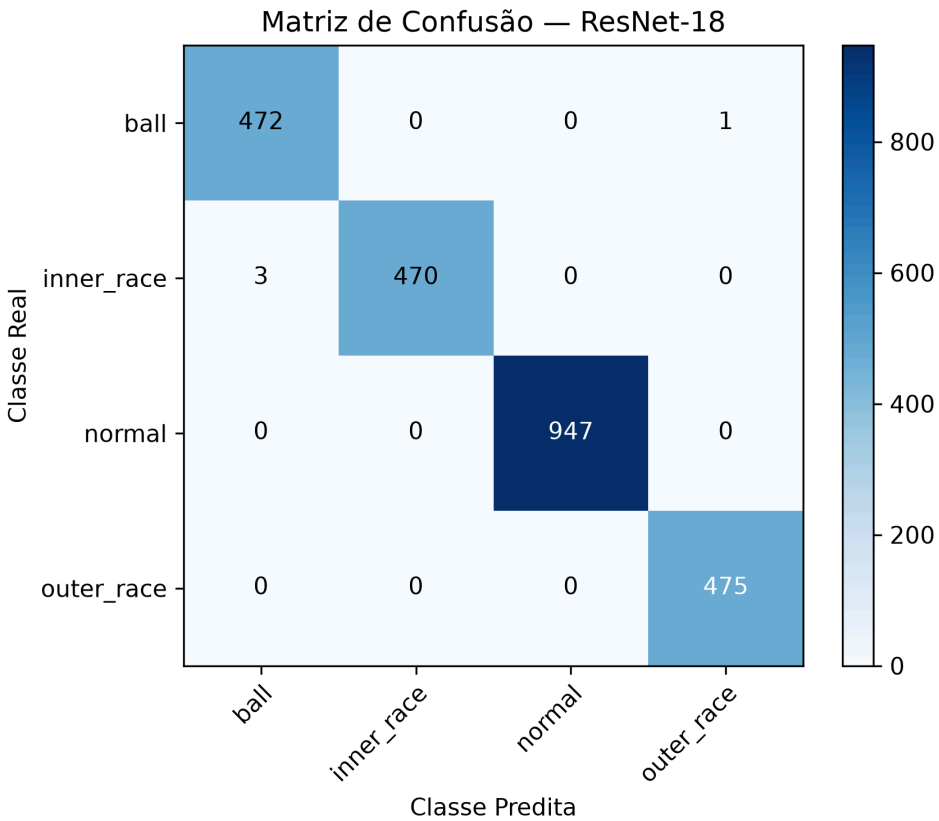
[!IMPORTANT] **Detecção Binária (100% de Precisão e Recall na condição Normal):** Na detecção de **presença de falha** (*Normal vs. Com Falha*), todos os modelos obtiveram **100.00% de Precisão e Recall** nas amostras de teste da classe normal. O pipeline alcançou **zero Falsos Negativos** (nenhum rolamento defeituoso passou como normal) e **zero Falsos Alarmes** (nenhum rolamento normal foi apontado com defeito). A pequena variação de acurácia (ex: 99.83% na ResNet-18) decorre exclusivamente da classificação específica entre os tipos de falha (*ball, inner_race, outer_race*).

[!NOTE] A divisão do dataset é realizada por **arquivos .mat inteiros** (e não por janelas individuais), garantindo que nenhuma amostra compartilhada por sobreposição (overlap de 50%) esteja presente nos conjuntos de treino e teste simultaneamente. Isso comprova a capacidade real de generalização dos modelos de Transfer Learning (até 99.83%) em comparação à CNN customizada (97.59%).

Curvas de Aprendizado (Perda e Acurácia — ResNet-18)
Curvas de Aprendizado — resnet18



Matriz de Confusão no Teste Inédito (ResNet-18 — 99.83% de Acurácia)



Estrutura do repositório

```
bearing-fault-diagnosis-cwt-cnn/
├── data/
│   ├── cwru/                # Dataset CWRU bruto (.mat) e processado
│   ├── paderborn/          # Dataset Paderborn (PU) bruto (.mat) e processado
│   └── order_tracking/      # Datasets reamostrados em Ângulo-Ordem (Order-CWT)
├── notebooks/              # Notebooks organizados por módulo de estudo
│   ├── 1_cwru/              # 1. Pipeline CWRU (Exploração, BearingCNN,
Transfer Learning)
│   ├── 2_paderborn/         # 2. Pipeline Paderborn (Exploração, BearingCNN,
Transfer Learning)
│   ├── 3_cross_domain/     # 3. Avaliação Cruzada e Análise de Domain Shift
│   └── 4_order_tracking/    # 4. Computed Order Tracking (Invariância à
Velocidade)
├── src/
│   └── config.py            # Configurações globais de ambos os datasets e
ordens
├── dataset_cwru.py          # Leitura e janelamento dos arquivos .mat do
CWRU
├── generate_dataset_cwru.py  # Geração do dataset de escalogramas do CWRU
├── dataset_paderborn.py     # Leitura e janelamento do dataset de Paderborn
(64 kHz)
├── generate_dataset_paderborn.py # Geração do dataset de escalogramas de
Paderborn
└── order_tracking.py        # Módulo de Computed Order Tracking (COT) e
Order-CWT
```

```
| | generate_order_dataset.py # Gerador de escalogramas Ângulo-Ordem
| | cwt_processor.py         # Cálculo da CWT e exportação dos escalogramas
224x224
| | visualization.py         # Geração de gráficos acadêmicos e curvas
| | cnn_processor.py         # Arquitetura BearingCNN e motor de treino
| | transfer_learning.py     # Construtores para ResNet18/Inception-
v3/EfficientNet-B0
| docs/
| | TCC.pdf                 # Relatório técnico completo do trabalho
| | images/                 # Figuras utilizadas neste README e na monografia
| requirements.txt
| LICENSE
| README.md
```

Instalação

```
git clone https://github.com/jvkauer/bearing-fault-diagnosis-cwt-cnn.git
cd bearing-fault-diagnosis-cwt-cnn

python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate

pip install -r requirements.txt
```

Dataset

Os dados brutos de vibração são fornecidos publicamente pela [CWRU Bearing Data Center](#). Baixe os arquivos `.mat` e posicione-os nas pastas correspondentes em `data/` (`ball`, `inner_race`, `normal`, `outer_race`) conforme indicado em `src/config.py`.

Uso

1. Gerar os escalogramas a partir dos sinais brutos:

```
python src/generate_dataset.py
```

2. Treinar o modelo de Transfer Learning via script ou notebook:

```
from src.transfer_learning import train_and_evaluate_transfer_model

resultado = train_and_evaluate_transfer_model(
    model_name="resnet18", # "resnet18" | "inception_v3" | "efficientnet_b0"
    num_epochs=10,
    lr=1e-3,
```

```
freeze=False # True para Feature Extraction, False para Fine-
Tuning
)

print(f"Acurácia no teste: {resultado['test_acc']:.2f}%")
```

O pipeline realiza checkpointing automático do melhor modelo (menor `val_loss`) e retorna histórico de treino, predições de teste e métricas para análise posterior.

Modelos de Transfer Learning

Arquitetura	Camadas	Parâmetros (total)	Característica principal
ResNet18	18	~11M	Blocos residuais (shortcut connections)
Inception-v3	42	~24M	Módulos paralelos com kernels de múltiplos tamanhos
EfficientNet-B0	variável	~5M	Escalonamento composto (profundidade/largura/resolução)

Todos os modelos são inicializados com pesos pré-treinados no ImageNet e adaptados ao problema por meio de feature extraction (backbone congelado) ou fine-tuning.

Base teórica

Projeto fundamentado em literatura consolidada de processamento de sinais e aprendizado profundo, incluindo os trabalhos de Boudiaf et al. (2016), Guo et al. (2018), Wang et al. (2019) e Kaya, Kuncan & Ertunç (2022) que reportam acurácias de até 99–100% combinando CWT com transfer learning. Os fundamentos matemáticos completos (FFT, STFT, CWT, arquiteturas de CNN) estão documentados no [Relatório Técnico \(TCC\)](#).

Roadmap

- ☒ Revisão teórica e definição da metodologia
- ☒ Pipeline de pré-processamento (CWT → escalogramas)
- ☒ CNN treinada do zero (**BearingCNN** - **97.59%** de acurácia no teste)
- ☒ Transfer Learning (**ResNet18**, **Inception-v3**, **EfficientNet-B0** - até **99.83%** de acurácia no teste)
- ☒ Comparação sistemática de resultados entre arquiteturas
- ☐ Validação em banco de dados de falhas reais (NASA IMS / Paderborn - TCC 2)

Autor

João Vitor Kauer Schuck

Engenharia de Computação — Universidade Federal de Pelotas (UFPEL)

[GitHub: jvkauer](#)

Licença

Distribuído sob a licença [MIT](#).